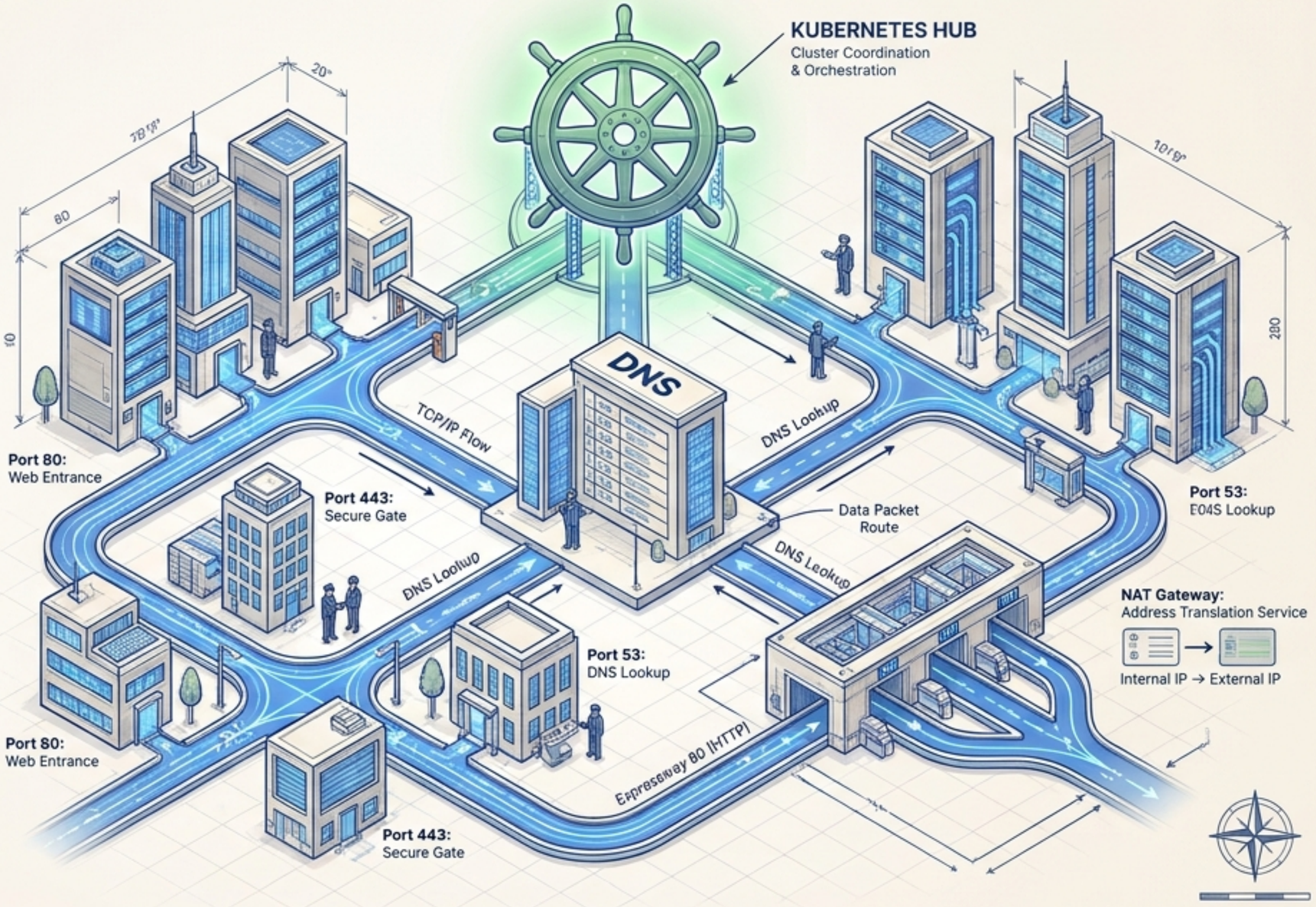
Redes para Iniciantes: O Guia Visual para Kubernetes

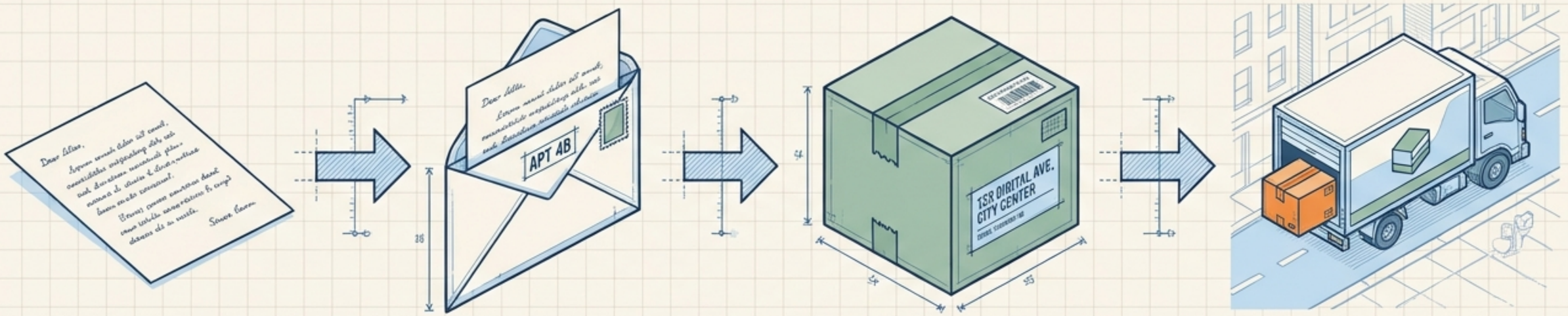
A logística da cidade digital: TCP/IP, DNS, Portas e NAT traduzidos para o dia a dia (e como tudo isso faz o seu cluster funcionar).





Gemini Notebook

O Modelo em Camadas (As Bonecas Russas da Internet)



Aplicação
(HTTP)

Transporte
(TCP + Porta)

Internet
(IP)

Enlace
(Ethernet/MAC)

Encapsulamento: Cada camada embrulha o trabalho da camada de cima e não se importa com o que tem dentro.

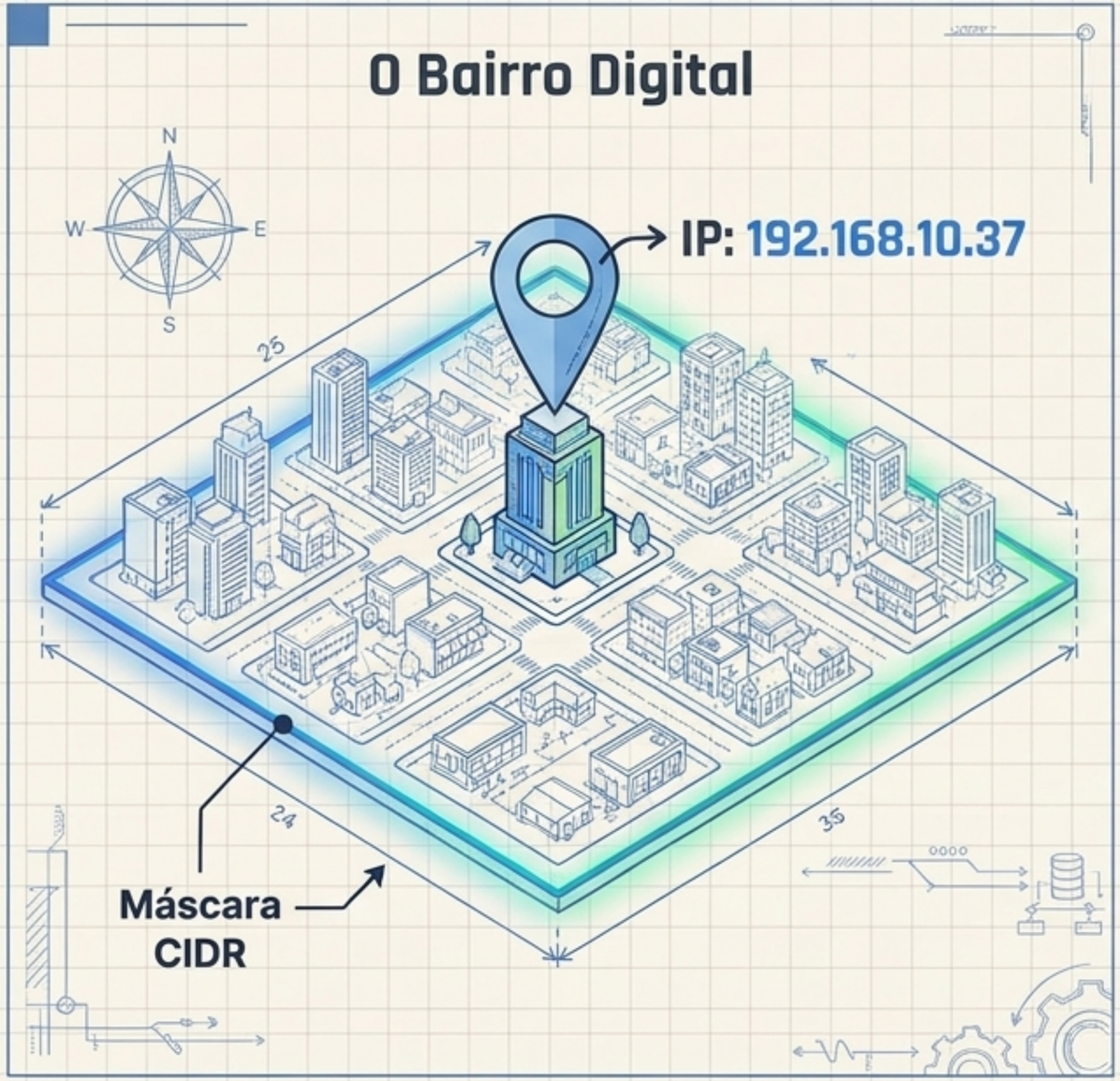
No Kubernetes

- ☑ **Services** (ClusterIP/NodePort) operam na Camada 4 (Lendo o envelope / Portas).
- ☑ O **Ingress** opera na Camada 7 (Lendo a carta / Caminho HTTP).

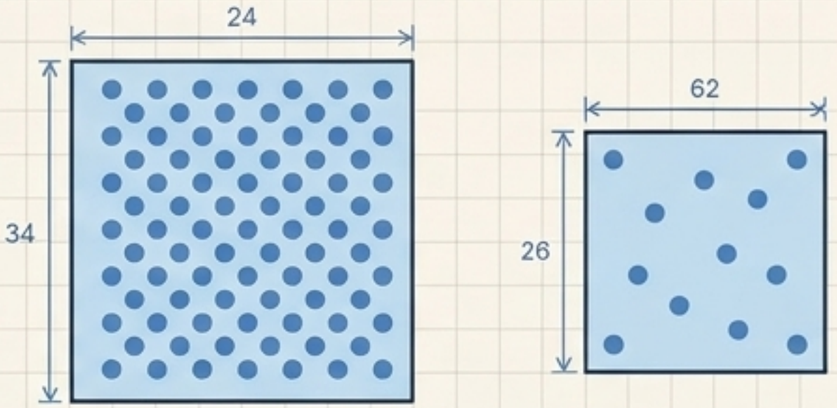
Gemini Notebook

Endereços IP e Máscaras (As Ruas e CEPs)

O Bairro Digital



Tamanho dos Bairros (CIDR)



/24 = Bairro grande
(254 hosts)

/26 = Quarteirão
(62 hosts)

IPs Privados
(10.x.x.x, 172.16.x.x, 192.168.x.x) são condomínios fechados. Não existem no mapa público da internet.

No Kubernetes

- [x] Todo cluster gerencia "bairros" (CIDRs) distintos. Um bloco exclusivo para os Pods (ex: 10.244.0.0/16) e outro separado para os Services (ex: 10.96.0.0/12). Nunca sobreponha estes com a rede da sua empresa!

Gemini Notebook

Portas, TCP e UDP (Os Apartamentos e a Entrega)

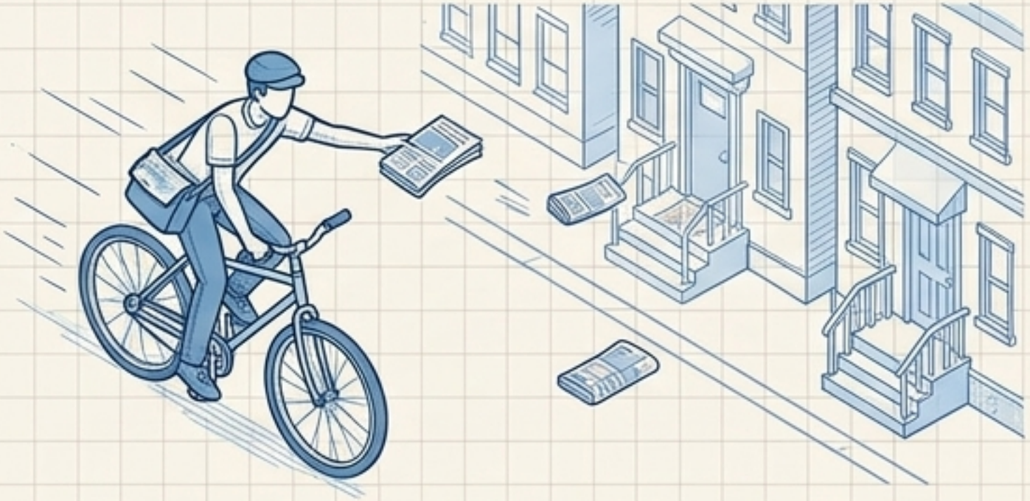
Portas (0 a 65535): O IP acha o prédio. A porta acha o apartamento (o programa rodando).

TCP (Correio Registrado)



Confiável, pacotes ordenados e com recibo de entrega.
Usado para HTTP, SSH e Bancos de Dados.

UDP (Panfletagem Rápida)



Rápido, sem conexão prévia e não confere se chegou.
Usado para DNS, Vídeo ao vivo e Jogos.

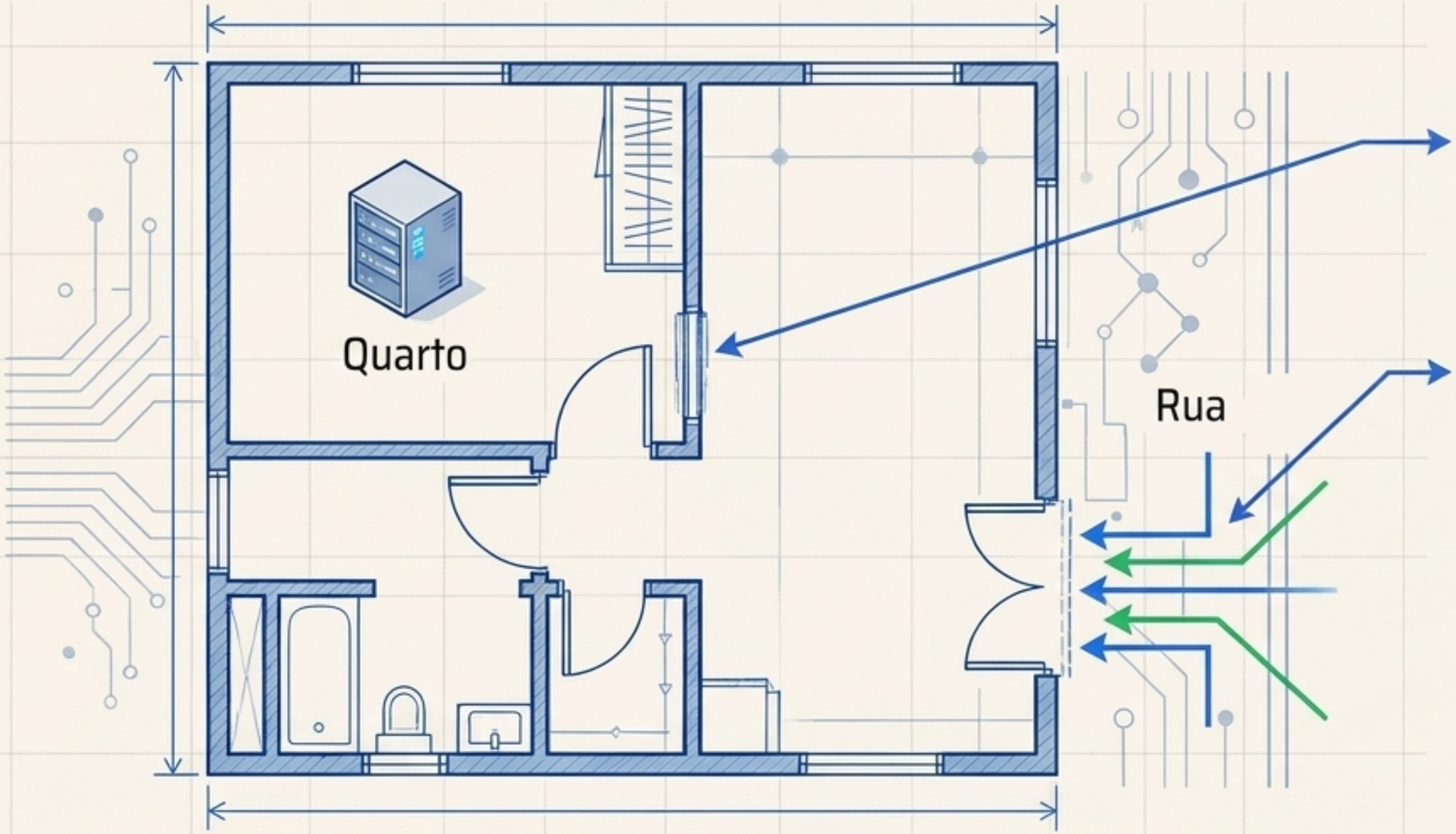
Use `ss -ltnp` no terminal para ver quem está escutando nas portas TCP.

No Kubernetes

- ☑ Decore estas portas vitais: 6443 (API Server do K8s) e 10250 (Kubelet nos nós).

Gemini Notebook

O Enigma do Endereço Local (127.0.0.1 vs 0.0.0.0)



127.0.0.1 (Localhost): Uma porta interna do quarto. O app escuta aqui, mas ninguém de fora consegue entrar.

0.0.0.0: A porta principal da rua. O aplicativo atende entregas vindo de qualquer lugar (todas as interfaces).

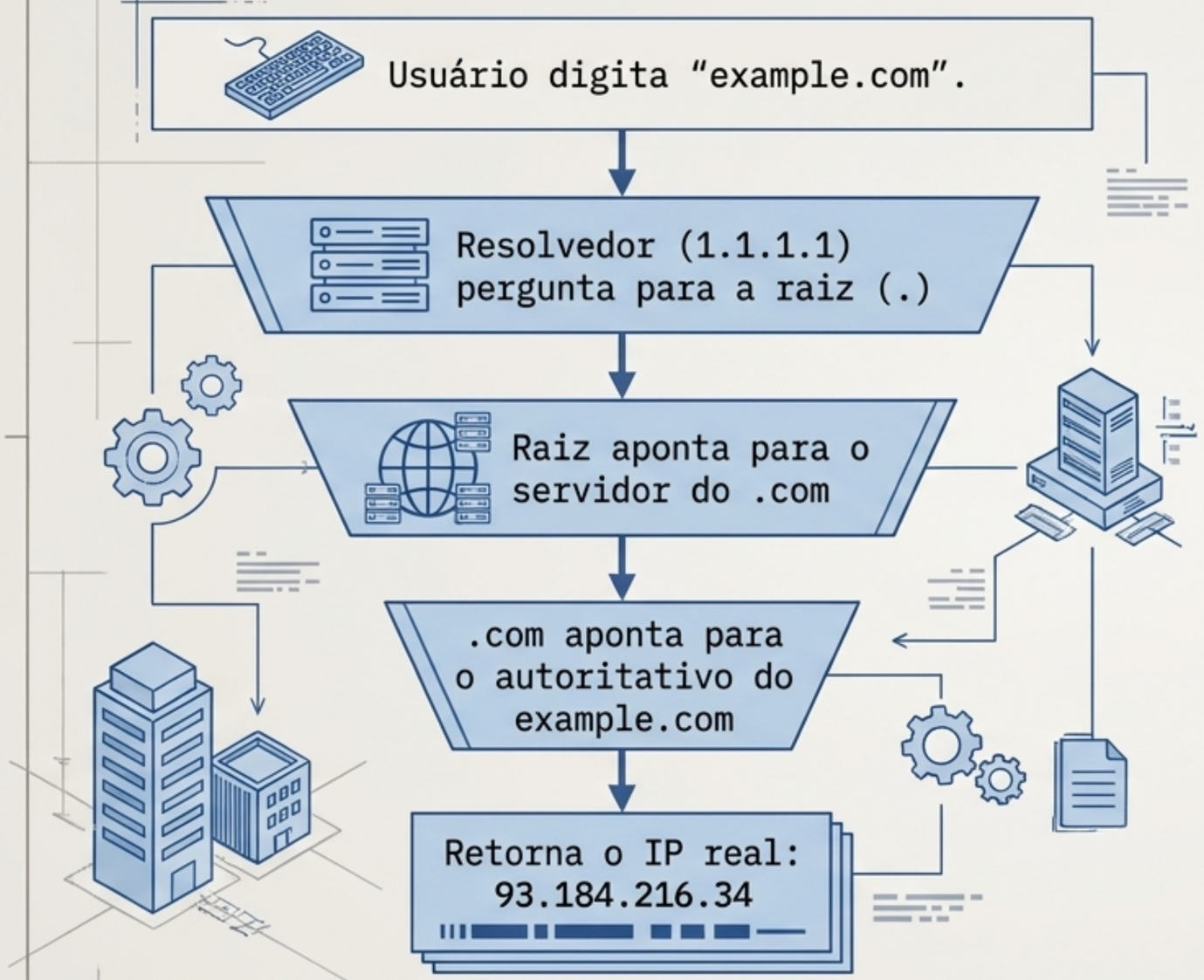
Sintoma clássico: "Funciona na minha máquina, **mas ninguém acessa!**" - Provavelmente seu app está escondido em **127.0.0.1**.

No Kubernetes

Se a sua aplicação dentro do Pod escutar apenas em `127.0.0.1`, o Service do Kubernetes não conseguirá alcançá-la de fora. Tem que ser configurada para `0.0.0.0`

Gemini Notebook

DNS (A Lista Telefônica da Internet)



Registros Chave

- `A`: Nome real ➔ IP (O clássico).
- `CNAME`: Apelido ➔ Nome verdadeiro (O redirecionador).

Use `dig example.com` para perguntar ao DNS.

No Kubernetes

O CoreDNS é a lista telefônica interna do condomínio. É por isso que um Pod acha o outro apenas pelo nome do Service (ex: meu-servico), sem precisar decorar IPs dinâmicos.

Gemini Notebook

HTTP / HTTPS (A Conversa)



Matriz de Status (O Semáforo)

2xx (Sucesso): "Toma aqui o que pediu" (ex: 200 OK).	**3xx (Redirecionamento):** "Mudou de endereço, vá para lá" (ex: 301).
4xx (Erro do Cliente): "Você pediu algo que não existe ou não tem acesso" (ex: 404, 403).	**5xx (Erro do Servidor):** "Eu pifei ou o banco de dados caiu" (ex: 502, 503).

`curl -i http://...` (Simula o navegador pelo terminal).

No Kubernetes

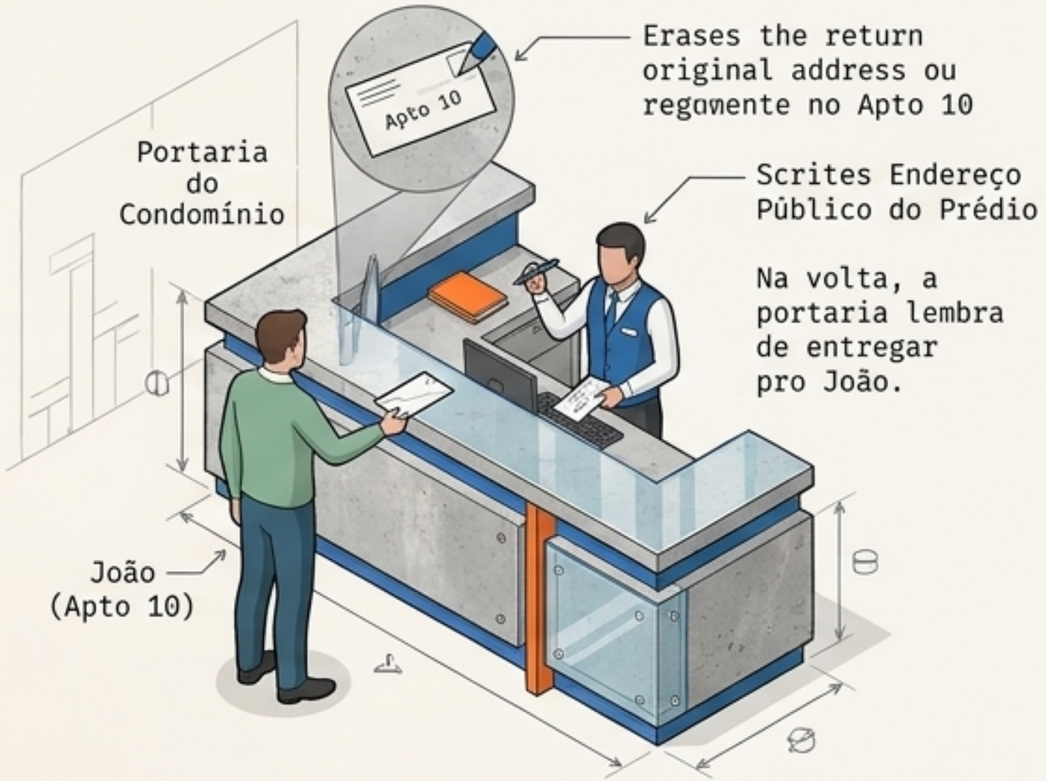
As **Probes de saúde (Liveness/Readiness)** testam o Pod chamando HTTP (esperando um código 200-399).

O próprio `kubectl get pods` é apenas um GET disfarçado!

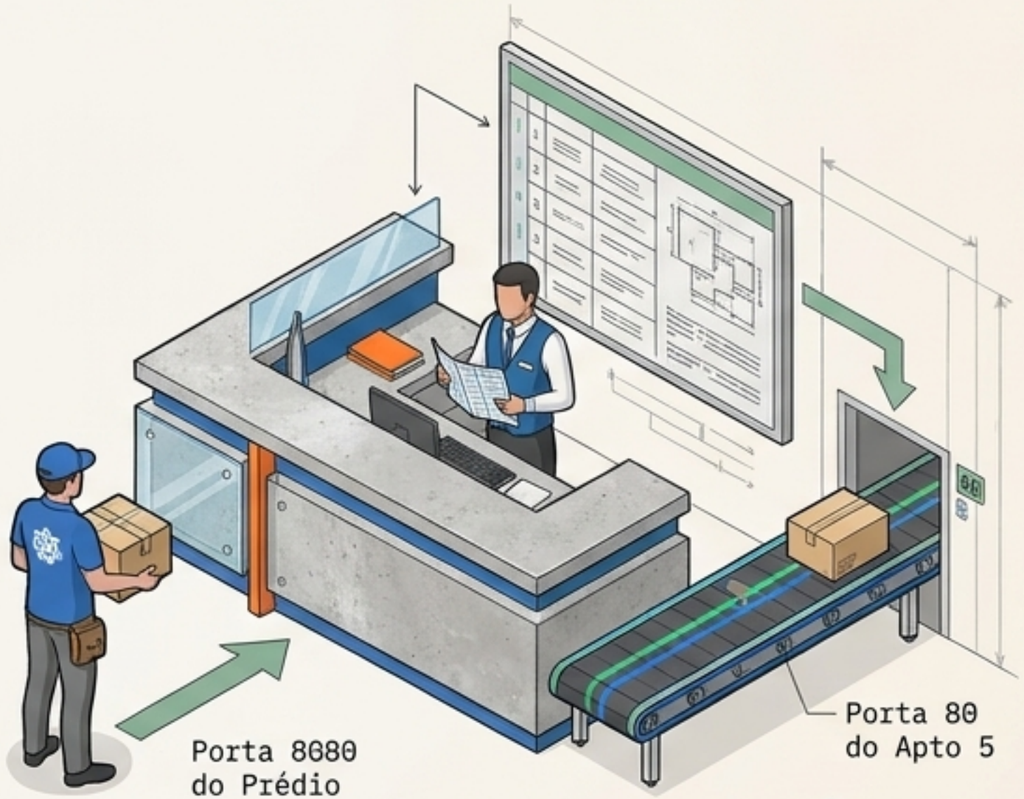
Gemini Notebook

NAT (A Sala de Correspondência)

SNAT (De dentro pra fora)



DNAT (De fora pra dentro - Redirecionamento)



No Kubernetes

É aqui que a mágica da rede acontece! O `kube-proxy` e os Services do tipo NodePort fazem DNAT o tempo todo, interceptando o IP do Serviço e trocando pelo IP do Pod correto nos bastidores.

Gemini Notebook

Firewall (O Porteiro com a Prancheta)



A regra de ouro

Negar tudo por padrão, permitir só o necessário (Default Deny). As regras são lidas de cima para baixo. A primeira que bater com o pacote é executada (Permitir ou Descartar).

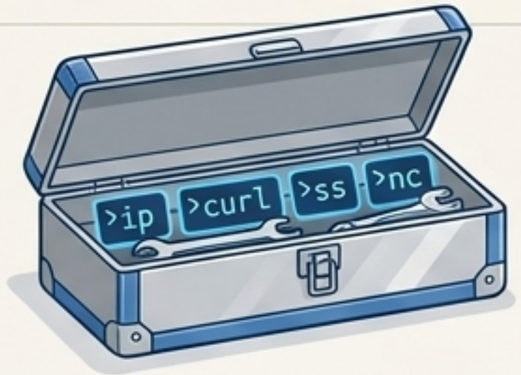
⚠ **Aviso de Sobrevivência: NUNCA** ative um firewall remoto (`ufw enable`) sem antes criar a regra que permite a porta 22 (SSH). Senão, você se tranca para fora do próprio servidor!

No Kubernetes

Isolamento de rede (NetworkPolicies) nada mais é do que regras de firewall declaradas em YAML, dizendo exatamente qual Pod tem permissão para falar com qual.

Gemini Notebook

A Caixa de Ferramentas CLI (Guia de Sobrevivência)



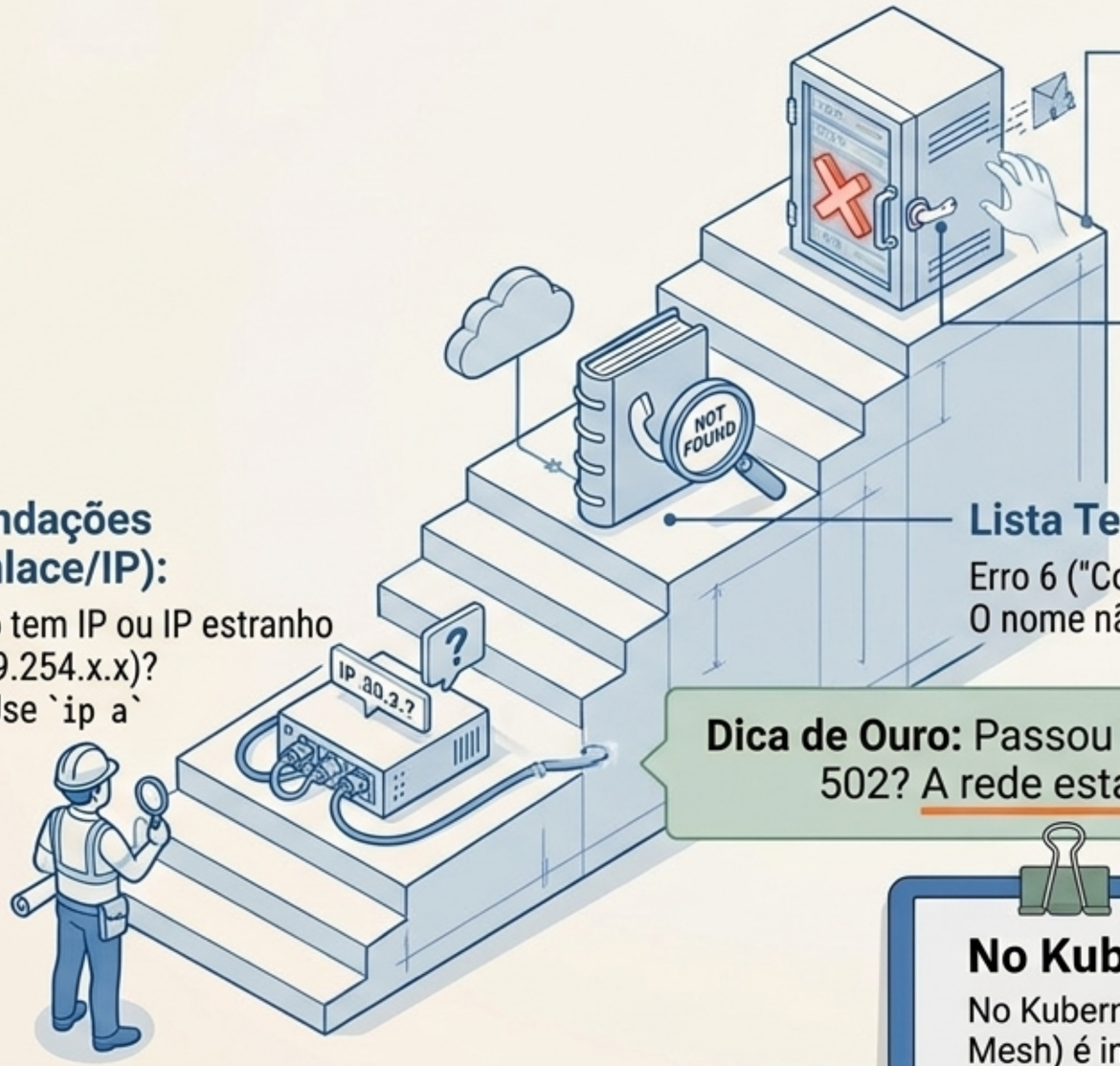
Comando	Para que serve	Analogia na Cidade
`ip -br a`	Qual é o meu endereço e máscara?	Olhar a placa na fachada do prédio.
`ip route`	Pra onde mando pacotes que não são daqui?	Achar a saída do bairro (Gateway).
`ss -ltnp`	Quem tá escutando aqui no servidor?	Bater nas portas dos apartamentos.
`curl -i`	Falar com a aplicação web.	Entregar e ler a carta (Requisição HTTP).
`nc -vz`	Testar se uma porta TCP está aberta.	Tentar girar a maçaneta pra ver se tá trancada.

No Kubernetes

Em containers e orquestradores como Kubernetes, essas ferramentas são cruciais para debugar a rede interna (overlay network) e entender como os Pods e Services se comunicam quando algo falha.

Gemini Notebook

Troubleshooting - A Escada do Diagnóstico



Porteiro (Firewall/Rota):

Erro 28 ("Timeout"). Pacote sumiu no caminho, foi bloqueado ou perdeu a rota. ➔ Use `traceroute` ou `ufw`



Maçaneta (Transporte):

Erro 7 ("Connection refused"). O IP responde, mas ninguém escuta na porta. ➔ Use `ss -ltnp`

Lista Telefônica (DNS):

Erro 6 ("Could not resolve host"). O nome não existe. ➔ Use `dig`

Fundações (Enlace/IP):

Não tem IP ou IP estranho (169.254.x.x)? ➔ Use `ip a`

Dica de Ouro: Passou ileso pelos 4 degraus e recebeu um Erro HTTP 502? A rede está perfeita. O problema é lá na aplicação!

No Kubernetes

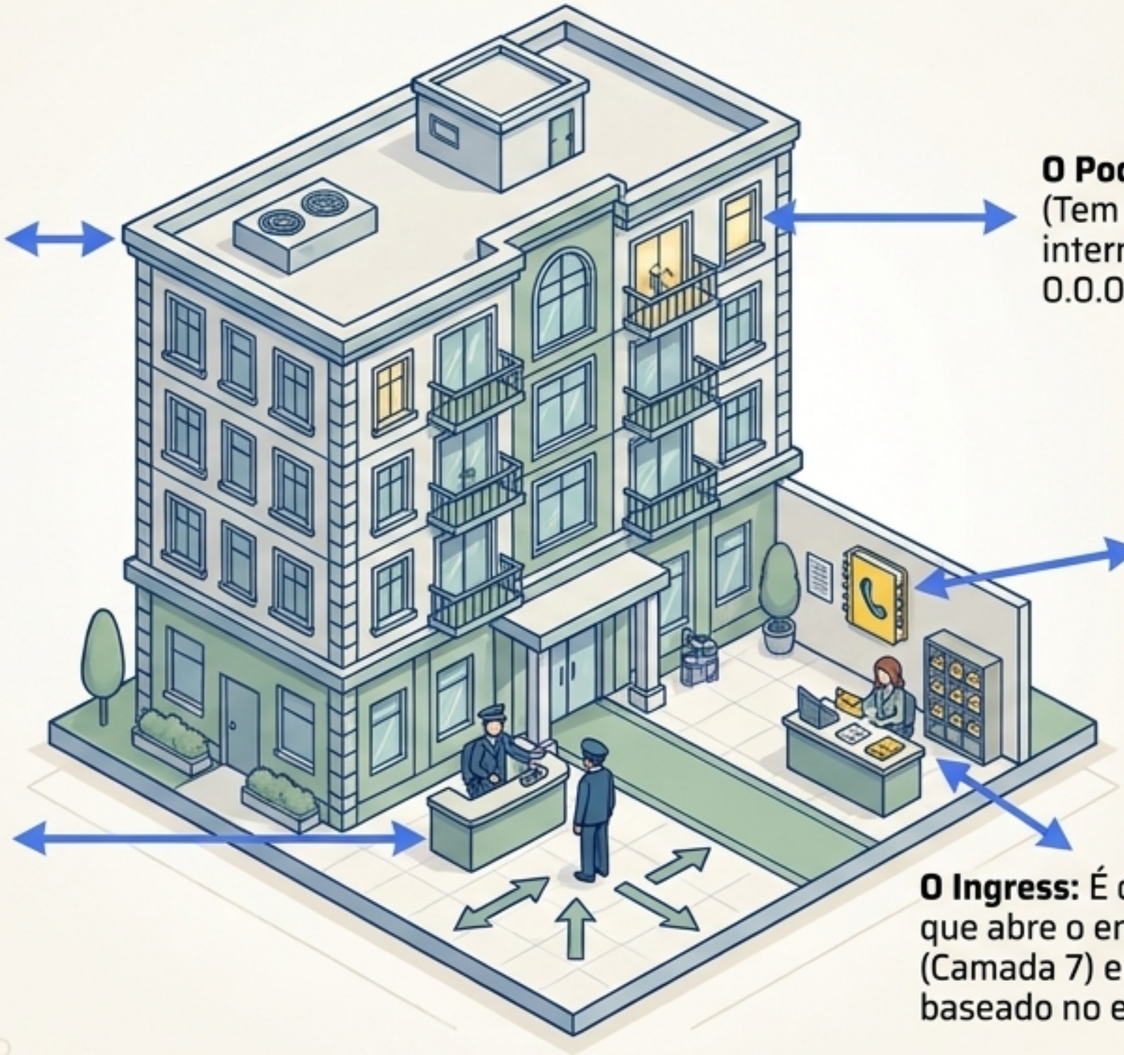
No Kubernetes, a visibilidade da rede (Network Policies e Service Mesh) é integrada. O diagnóstico se torna mais sobre verificar regras e configurações declarativas do que rastrear pacotes manualmente.

Gemini Notebook

Síntese - O Mapa do Kubernetes (Aha! Moment)

O Nó (Node):
É o Prédio inteiro
(Endereço IP público/privado).

O Pod: É o Apartamento
(Tem seu próprio IP no CIDR interno, rodando app na porta 0.0.0.0).



O CoreDNS: É a Lista Telefônica pendurada no hall do prédio.

O Service / NodePort: É o Porteiro fazendo NAT
(Redirecionando tráfego externo para o Pod correto).

O Ingress: É o Recepcionista que abre o envelope HTTP (Camada 7) e decide o caminho baseado no endereço web.

De Peças Soltas a um Cluster: Tudo o que você aprendeu é a engrenagem que faz o Kubernetes respirar. Redes não é magia, é pura logística.

No Kubernetes

Em containers e orquestradores como Kubernetes, essas ferramentes são cruciais para debugar a rede interna (overlay network) e entender como os Pods e Services se comunicam quando algo falha. A visibilidade da rede (Network Policies e Service Mesh) é integrada.

Gemini Notebook

Checklist do Iniciante (Você aprendeu!)

- [x] Entendo que TCP confirma entregas e UDP apenas "joga" o pacote.
- [x] Sei que 127.0.0.1 é uma porta trancada para o mundo exterior.
- [x] O DNS converte nomes fáceis em IPs complexos.
- [x] O NAT e os Firewalls são os porteiros que mascaram e filtram conexões.
- [x] Diagnóstico de rede se faz sem desespero: camada por camada, sempre de baixo para cima.

No Kubernetes

A base está sólida. No Kubernetes, essa lógica de rede é abstraída e declarativa.

O foco muda de 'como conectar' para 'o que deve estar conectado' através de configurações, simplificando a gestão em escala. Mas os princípios fundamentais permanecem os mesmos.

Pronto para parar de digitar comandos manuais e começar a escrever essa logística como código?

Próximo Passo: Módulo 6 - YAML e JSON. A linguagem universal do Kubernetes.

Gemini Notebook